

# Programación para Física y Astronomía

Departamento de Física.

---

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Ejercicios Prácticos Integrados

Soluciones de Referencia

Consolidación y Retroalimentación

Conclusiones

Apéndice: Ver también

# Introducción y Conexión

---

- Ajuste lineal con **curve\_fit**; residuos,  $R^2$ ,  $\chi^2_{\text{red}}$ .
- Ejemplo: Ley de Hubble  $v = H_0 d + b$  en datos simulados.
- Interpolación 1D: lineal vs cúbica; evitar extrapolación.
- Ajuste de distribuciones y modelo elegido con AIC/KS.

## Objetivo de hoy

Aplicar y consolidar estos conceptos en 6 ejercicios cortos y guiados.

- Consolidar el uso de `scipy.optimize.curve_fit` y diagnóstico básico.
- Aplicar `interp1d` (lineal/cúbica) en señales con huecos.
- Elegir distribuciones con `scipy.stats` mediante AIC y KS.

# Ejercicios Prácticos Integrados

---

# Ejercicios Prácticos Integrados $\ni$ Ejercicio 1:

## Calibración lineal sin gráficos



### Enunciado

- Simule 12 medidas  $(T, V)$  de un sensor  $V = aT + b$  con  $\sigma_V$  constante.
- Estime  $a, b$  con `curve_fit`. Reporte  $R^2$  y  $\chi^2_{\text{red}}$ .
- Opcional fuera de clase: graficar (omitido aquí).

**Física:** sensibilidad ( $V/^\circ\text{C}$ ) y offset del sensor.

# Ejercicios Prácticos Integrados $\ni$ Ejercicio 2:

## Hubble con diagnóstico básico



### Enunciado

- Genere 25 galaxias con  $d \in [10, 200]$  Mpc y  $\sigma_v = 180$  km/s constante.
- Ajuste  $v = H_0 d + b$  con `curve_fit`.
- Calcule  $\chi^2_{\text{red}}$  y reporte  $H_0$  con su error.
- Grafique datos, ajuste y residuos.

**Física:** estimación básica de la constante de Hubble.



# Ejercicios Prácticos Integrados $\ni$ Ejercicio 3:

## Interpolación de curva de luz



### Enunciado

- Simule  $F(t) = 1 + 0.1 \sin(2\pi t/P)$  en  $[0,10]$  con 60 puntos.
- Elimine 20 puntos al azar.
- Interpole con lineal y cúbica.
- Grafique las tres curvas (original, lineal, cúbica) y comente diferencias.

**Nota:** enfoque visual, sin calcular integrales.

# Ejercicios Prácticos Integrados $\ni$ Ejercicio 4:

## Datos positivos: Lognormal vs Normal



### Enunciado

- Genere 1500 datos positivos tipo lognormal ( $\mu = 0, \sigma = 0.6$ ).
- Ajuste distribuciones Lognormal y Normal.
- Calcule AIC para ambas y grafique histograma + PDFs.
- Concluya cuál modelo es más apropiado.

**Objetivo:** selección de modelo con criterio cuantitativo simple.

## Soluciones de Referencia

---

# Soluciones de Referencia $\ni$ Solución 1:

## Calibración lineal



```
1 import random, math
2 from scipy.optimize import curve_fit
3
4 # Generar datos sintéticos del sensor
5 T = [i*10 for i in range(12)] # Temperaturas de 0 a 110°C
6 a_true, b_true, sigma = 0.018, 0.15, 0.02
7 V = [a_true*t + b_true + random.gauss(0.0, sigma) for t in T]
8
9 # Definir modelo lineal
10 def modelo_lineal(x, a, b):
11     return a*x + b
12
13 # Ajustar con curve_fit
14 sig = [sigma]*len(T)
15 popt, _ = curve_fit(modelo_lineal, T, V, sigma=sig, absolute_sigma=True)
16 a_est, b_est = popt
17
18 # Calcular métricas de calidad
19 V_fit = [modelo_lineal(t, a_est, b_est) for t in T]
20 media_V = sum(V)/len(V)
21 SS_res = sum((vi - vfi)**2 for vi, vfi in zip(V, V_fit))
22 SS_tot = sum((vi - media_V)**2 for vi in V)
23 R2 = 1 - SS_res/SS_tot
24 chi2_red = sum(((vi - vfi)/sigma)**2 for vi, vfi in zip(V, V_fit))/(len(T)-2)
25
26 print(f"Sensibilidad: a={a_est:.4f} V/°C")
27 print(f"Offset: b={b_est:.3f} V")
28 print(f"R²={R2:.3f},  $\chi^2_{red}$ ={chi2_red:.2f}")
```

Lectura:  $\chi^2_{red} \approx 1$  confirma que  $\sigma$  estimado es razonable

# Soluciones de Referencia $\ni$ Solución 2:

## Hubble con diagnóstico



```
1 import random, math
2 from scipy.optimize import curve_fit
3 import matplotlib.pyplot as plt
4
5 # Generar datos de galaxias
6 H0_true, sigma_v = 70.0, 180.0
7 n = 25
8 d = [10 + i*(190/(n-1)) for i in range(n)] # 10 a 200 Mpc
9 v_obs = [H0_true*d + random.gauss(0.0, sigma_v) for di in d]
10
11 # Ajustar modelo de Hubble
12 def hubble(d, H0, b):
13     return H0*d + b
14
15 sig = [sigma_v]*n
16 popt, pcov = curve_fit(hubble, d, v_obs, sigma=sig, absolute_sigma=True)
17 H0_est, b_est = popt
18 perr = [math.sqrt(pcov[i][i]) for i in range(len(popt))]
19
20 # Calcular chi2_red
21 v_fit = [hubble(di, H0_est, b_est) for di in d]
22 res = [vi - vfi for vi, vfi in zip(v_obs, v_fit)]
23 chi2_red = sum((ri/sigma_v)**2 for ri in res)/(n-2)
24
25 print(f"H0 = {H0_est:.1f}  $\pm$  {perr[0]:.1f} km/s/Mpc")
26 print(f"Intercepto b = {b_est:.1f} km/s")
27 print(f" $\chi^2_{red}$  = {chi2_red:.2f}")
```

# Soluciones de Referencia $\ni$ Solución 2:

## Hubble con diagnóstico (gráficos)



```
25 # Graficar datos y ajuste
26 plt.figure(figsize=(10, 4))
27
28 # Panel 1: Datos y ajuste
29 plt.subplot(1, 2, 1)
30 plt.scatter(d, v_obs, s=30, alpha=0.6, label='Galaxias')
31 plt.plot(d, v_fit, 'r-', linewidth=2, label=f'H0={H0_est:.1f}')
32 plt.xlabel('Distancia (Mpc)')
33 plt.ylabel('Velocidad (km/s)')
34 plt.legend()
35 plt.grid(True, alpha=0.3)
36
37 # Panel 2: Residuos
38 plt.subplot(1, 2, 2)
39 plt.scatter(d, res, s=30, alpha=0.6)
40 plt.axhline(0, color='r', linestyle='--')
41 plt.xlabel('Distancia (Mpc)')
42 plt.ylabel('Residuos (km/s)')
43 plt.grid(True, alpha=0.3)
44
45 plt.tight_layout()
46 plt.show()
```

**Física:** residuos reflejan velocidades peculiares de las galaxias.

# Soluciones de Referencia $\ni$ Solución 3:

## Interpolación de curva de luz



```
1 from scipy.interpolate import interp1d
2 import random, math
3 import matplotlib.pyplot as plt
4
5 # Generar curva de luz completa
6 P = 2.5 # periodo
7 t_full = [i*0.05 for i in range(201)] # 0 a 10 en 201 puntos
8 F_full = [1.0 + 0.1*math.sin(2*math.pi*t/P) for t in t_full]
9
10 # Eliminar 20 puntos al azar
11 random.seed(42)
12 indices = random.sample(range(len(t_full)), len(t_full) - 20)
13 indices.sort()
14 t_obs = [t_full[i] for i in indices]
15 F_obs = [F_full[i] for i in indices]
16
17 # Crear interpoladores
18 f_lin = interp1d(t_obs, F_obs, kind='linear')
19 f_cub = interp1d(t_obs, F_obs, kind='cubic')
20
21 # Evaluar en grilla fina
22 t_interp = [i*0.025 for i in range(401)] # más fino para suavidad
23 F_lin = [f_lin(t) for t in t_interp]
24 F_cub = [f_cub(t) for t in t_interp]
```

# Soluciones de Referencia $\ni$ Solución 3:

## Interpolación (gráficos)



```
23 # Graficar comparación
24 plt.figure(figsize=(10, 6))
25 plt.plot(t_full, F_full, 'k-', alpha=0.3, linewidth=2, label='Original')
26 plt.scatter(t_obs, F_obs, s=20, c='black', zorder=5, label='Observaciones')
27 plt.plot(t_interp, F_lin, 'b--', linewidth=1.5, label='Lineal')
28 plt.plot(t_interp, F_cub, 'r-', linewidth=1.5, label='Cúbica')
29 plt.xlabel('Tiempo')
30 plt.ylabel('Flujo F(t)')
31 plt.title('Interpolación de Curva de Luz')
32 plt.legend()
33 plt.grid(True, alpha=0.3)
34 plt.show()
35
36 # Comentario: La interpolación cúbica sigue mejor la forma suave
37 # de la señal original, mientras que la lineal produce segmentos
38 # rectos entre puntos. Ambas son exactas en los puntos observados.
```

**Nota:** cúbica captura mejor la variación suave; lineal es más robusta al ruido.



# Soluciones de Referencia $\ni$ Solución 4:

## Lognormal vs Normal



```
1 from scipy import stats
2 import random
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 # Generar datos lognormales
7 random.seed(42)
8 n = 1500
9 mu, sigma = 0.0, 0.6
10 x = [random.lognormvariate(mu, sigma) for _ in range(n)]
11
12 # Ajustar distribución Lognormal
13 s_log, _, scale_log = stats.lognorm.fit(x, floc=0)
14 ll_log = sum(stats.lognorm.logpdf(xi, s_log, 0, scale_log) for xi in x)
15 k_log = 2 # parámetros: s y scale
16 AIC_log = 2*k_log - 2*ll_log
17
18 # Ajustar distribución Normal
19 mu_norm, sig_norm = stats.norm.fit(x)
20 ll_norm = sum(stats.norm.logpdf(xi, mu_norm, sig_norm) for xi in x)
21 k_norm = 2 # parámetros: mu y sigma
22 AIC_norm = 2*k_norm - 2*ll_norm
23
24 print(f"Lognormal: AIC = {AIC_log:.1f}")
25 print(f"Normal: AIC = {AIC_norm:.1f}")
26 print(f"Mejor modelo: {'Lognormal' if AIC_log < AIC_norm else 'Normal'}")
```

# Soluciones de Referencia $\ni$ Solución 4:

## Lognormal vs Normal (gráficos)



```
24 # Graficar histograma y PDFs ajustadas
25 plt.figure(figsize=(10, 6))
26 plt.hist(x, bins=40, density=True, alpha=0.5, edgecolor='black',
27          label='Histograma')
28
29 # Generar curvas de densidad
30 x_plot = np.linspace(0, max(x), 500)
31 pdf_log = stats.lognorm.pdf(x_plot, s_log, 0, scale_log)
32 pdf_norm = stats.norm.pdf(x_plot, mu_norm, sig_norm)
33
34 plt.plot(x_plot, pdf_log, 'r-', linewidth=2,
35          label=f'Lognormal (AIC={AIC_log:.1f})')
36 plt.plot(x_plot, pdf_norm, 'b--', linewidth=2,
37          label=f'Normal (AIC={AIC_norm:.1f})')
38
39 plt.xlabel('Valor')
40 plt.ylabel('Densidad de Probabilidad')
41 plt.title('Comparación de Ajustes: Lognormal vs Normal')
42 plt.legend()
43 plt.grid(True, alpha=0.3)
44 plt.show()
```

**Conclusión:** Lognormal tiene menor AIC y captura mejor la asimetría positiva.

## Consolidación y Retroalimentación

---

# Consolidación y Retroalimentación $\ni$ Puesta en Común - ¿Cómo les fue?

- ¿Dónde costó más: ajuste, interpolación o AIC/KS?
- ¿Qué indicadores reportarían siempre? ( $R^2$ ,  $\chi^2_{\text{red}}$ , AIC, KS)
- ¿Qué harían distinto si tuvieran datos reales con outliers?

## Objetivo

Construir criterio físico-estadístico al elegir modelos.

# Conclusiones

---

- Consolidamos ajuste lineal con `curve_fit` y diagnóstico básico.
- Aplicamos interpolación lineal/cúbica con integración.
- Seleccionamos distribuciones usando AIC y validamos con KS.

### Habilidad adquirida

Resolver y justificar modelos simples con métricas claras.

- Repasar: `curve_fit`, `interp1d`, `scipy.stats`.
- Práctica: repetir Ej. 5–6 con otro tamaño de muestra.

¡Nos vemos en la próxima sesión!

Apéndice: Ver también

---



## Apéndice: Ver también $\ni$ Extra: `curve_fit` lineal con errores

```
1 from scipy.optimize import curve_fit
2 import random, math
3
4 f = lambda x,m,b: m*x + b
5 x = [i*(10/29) for i in range(30)]
6 y = [f(xi, 2.0, 1.0) + random.gauss(0,0.5) for xi in x]
7 sigma = [0.5]*len(x)
8 popt, pcov = curve_fit(lambda x,m,b: [f(xi,m,b) for xi in x],
9                         x, y, sigma=sigma, absolute_sigma=True)
10 perr = [math.sqrt(pcov[i][i]) for i in range(len(popt))]
11 print(f"m={popt[0]:.2f}±{perr[0]:.2f}, b={popt[1]:.2f}±{perr[1]:.2f}")
```

## Apéndice: Ver también $\ni$ Extra: Prueba KS (solo referencia)

```
1 from scipy import stats
2 import random
3
4 x = [random.gauss(0,1) for _ in range(1000)]
5 mu, sigma = stats.norm.fit(x)
6 D, p = stats.kstest(x, 'norm', args=(mu, sigma))
7 print(f"KS D={D:.3f}, p={p:.3f} (no usar en P2 como criterio principal)")
```